

ITU-MiniTwit — BSc DevOps, Software Evolution and Software Maintenance

Group: BSc_group_m

Repository: <https://github.com/RonoITU/itu-devops2026-jackhammers>

Issue tracker: <https://github.com/RonoITU/itu-devops2026-jackhammers/issues>

Monitoring dashboard: <http://178.104.27.224:3000/d/chirp-aspnet-001/windysquirrels-monitoring->

Logging dashboard: <http://178.104.27.224:3000/d/app-logging-dashboard/windysquirrels-logging-d>

Name	ITU ID
Christian Philip Jørgensen	chpj@itu.dk
Jakob Sønder	jakso@itu.dk
Jacob Sponholtz	spon@itu.dk
Ronas Jacob Coban Olsen	rono@itu.dk
Rasmus Alexander Christiansen	ralc@itu.dk

1. System's Perspective

1.1 Design and Architecture

Author(s): Rasmus

This section describes the overall system architecture of MiniTwit, including how the application is structured, deployed, and monitored. The diagram above provides a visual overview of the components and their interactions across the infrastructure.

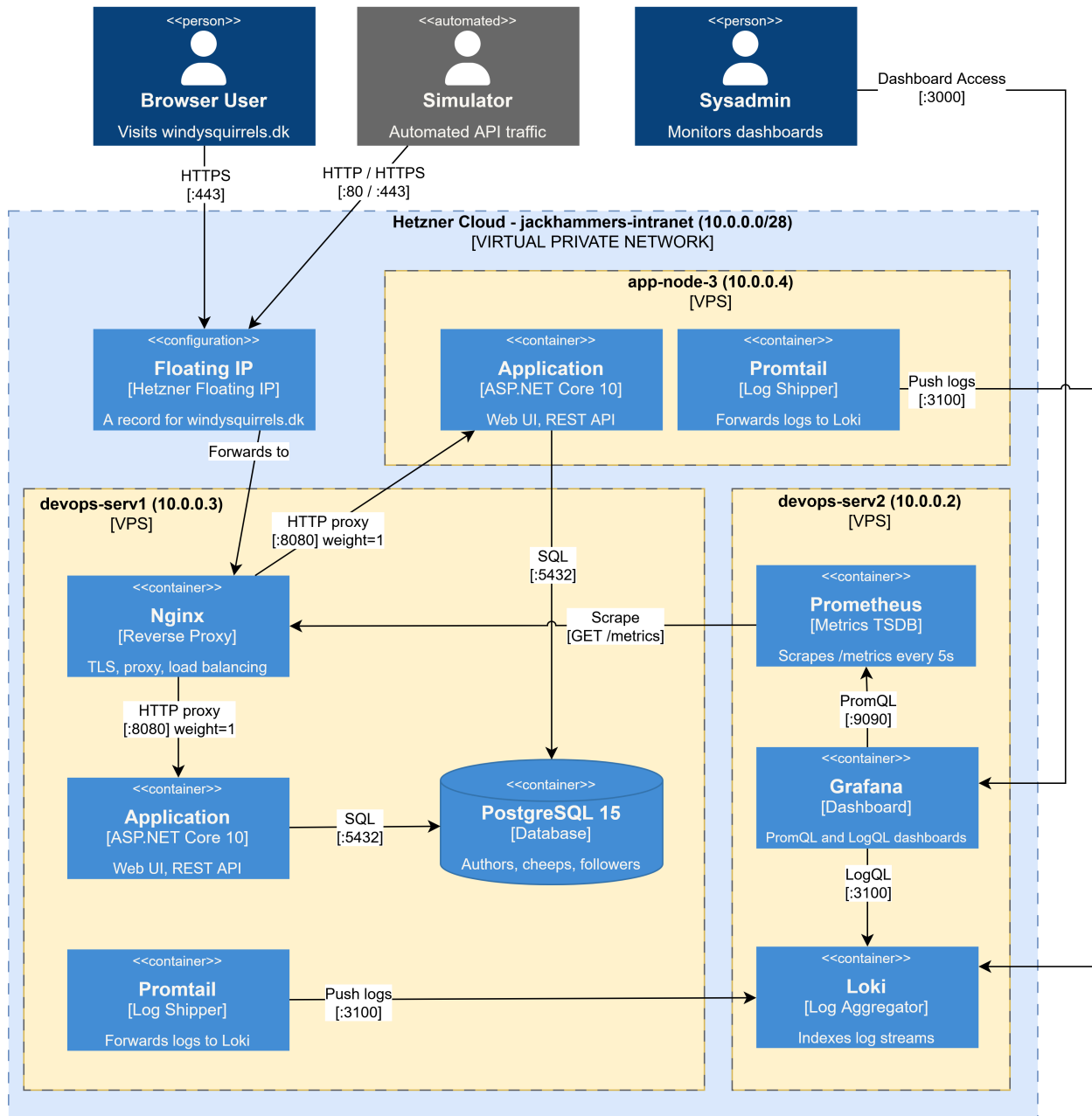


Figure 1: Diagram of the system architecture

MiniTwit is deployed across three Hetzner Cloud VPS nodes connected via a virtual private network

(10.0.0.0/28).

devops-serv1 (10.0.0.3) is the primary node. Nginx runs here as the sole public entry point, handling TLS termination via Let's Encrypt, redirects HTTP traffic to HTTPS, and weighted load balancing across both app nodes. The primary application instance serves web UI and REST API traffic and exposes a /metrics endpoint for Prometheus. PostgreSQL 15 hosts the shared database for all application data. Promtail runs as a log shipping agent, collecting Docker container logs from this node and forwarding them to Loki on devops-serv2.

app-node-3 (10.0.0.4) is the secondary application node. It runs an application replica that receives the majority of traffic from Nginx (weight=2) and connects to the shared PostgreSQL database on devops-serv1. Promtail runs here as well, shipping container logs to Loki on devops-serv2.

devops-serv2 (10.0.0.2) is dedicated to observability. Prometheus scrapes the /metrics endpoint on devops-serv1 every five seconds and stores the resulting time-series data. Loki aggregates the structured log streams pushed by the Promtail agents on devops-serv1 and app-node-3. Grafana provides dashboards over both data sources using PromQL for metrics and LogQL for logs.

1.2 Dependencies

Author(s):

Technology / Tool	Version	Purpose

1.3 Current State of the System

Author(s):

2. Process' Perspective

2.1 CI/CD Pipeline

Author(s):

2.2 Monitoring

Author(s):

2.3 Logging

Author(s):

2.4 Security Hardening

Author(s):

2.5 Availability and Scaling

Author(s):

3. Reflection Perspective

3.1 Evolution and Refactoring

Author(s):

3.2 Operation

Author(s):

3.3 Maintenance

Author(s):

3.4 DevOps Reflection

Author(s):

4. Use of Generative AI

Author(s): Rasmus

GitHub Copilot was used to assist with boilerplate code generation throughout development. It provided inline auto-completion that often matched the intended behavior being implemented, and helped untangle unfamiliar functionality across the broader codebase. This sped up the coding process significantly, particularly for repetitive or structurally predictable code.

Microsoft Copilot was used for general consultation when extending the infrastructure, serving as a quick reference for exploring options and approaches before committing to a direction.

Claude was used for more in-depth technical discussions, typically involving specific code examples. It helped provide clarity on complex topics and was particularly useful when a concept required more thorough explanation than a quick search could offer.